

SemPointer: An Indirection Layer over Dense Retrieval

Viksit Garg¹

Department of Artificial Intelligence & Machine Learning, Dr. Akhilesh Das Gupta
Institute of Professional Studies, New Delhi, India
viksitg427@gmail.com

Abstract. Long-context inference feeds every token to attention, so cost grows with length. SemPointer proposes semantic indirection: blocks sit in an off-attention store while short pointers stay in the prompt; a dense encoder retrieves blocks by address and resolves them to text. This concept paper gives an indirection model, a pointer design with routing and fallback, a cost-quality theory, and testable predictions for precision, fallback, and breakeven length. Honest thesis: the instantiation ties dense retrieval-augmented generation at higher token cost, and pointers are empirically inert. Scope: concept only; no fidelity or superiority claims.

Keywords: long-context inference, dense retrieval, semantic addressing, negative results, benchmarking

1 Introduction

2 Introduction

Dense retrieval rebuilds context per query: retrieval, ranking, and construction repeat when needs overlap, with passages used once and discarded, with no handle to keep, share, or update. Indirection separates what is retrieved from how it is reused: retrieval output should be addressable through persistent semantic pointers stored, compared, and dereferenced across queries, not rebuilt as flat text each time.

Analogy: indirection as OS pointers. A program holds a stable address for a movable, shareable object, resolved on need. Likewise, a pointer names reusable evidence, resolved late into prompt text with identity, sharing, and update.

RQ1 asks when pointer-mediated assembly preserves quality vs. standard concatenation; RQ2 asks whether pointers enable reuse across queries sharing evidence, in storage, tokens, effort; RQ3 asks what minimal machinery keeps pointers valid as index, corpus, query distribution shift. Assembly works: retrieve-order-concatenate yields coherent prompts on suites with $n=180$ and $n=1150$ for retrieval and $n=199$ for end-to-end quality. Pointers remain inert: dereferencing ties retrieval-augmented generation at higher cost, no reuse. Contribution is conceptual: mechanism, conditions where it could help, and measurements deciding failure or success.

3 Related Work

Hard prompt compression. LLMingua filters by small-LM perplexity under a budget [7]. Readable but single-digit; aggressive dropping breaks names, numbers, chains.

Soft gist tokens. Gist tuning compresses instructions [14]; ICAE autoencodes long context [5]. 500xCompressor reaches 480x at 62–73% capability training 0.3% of parameters [11]. Opaque decoder-specific slots: no inspection, quotation, composition without re-encoding.

KV-cache eviction and retrieval. SnapKV selects salient heads/positions [10]; H2O keeps heavy hitters. Quest loads top- k pages for 7x speedup [18]; RetrievalAttention pairs near-query search with local windows [12]. CacheBlend recomputes tokens to repair cross-chunk attention for 2.2–3.3x faster time-to-first-token [23]. Evicted detail is unrecoverable; cross-block composition needs recompute or refetch.

Agent memory systems. MemGPT pages history [16]; Mem0 adds ADD/UPDATE/DELETE over facts [4]; Zep builds a temporal knowledge graph [17]. State persists, but handles are off-prompt, unbudgeted, non-verbatim.

Landmark routing. Landmark attention routes through block markers [13]: random access as trained gates, not textual addresses to display, copy, or recombine.

Narrowed gap. No prior line combines: (i) prompt-resident textual handles with fixed per-block cost, (ii) verbatim resolution of the selected block, and (iii) a composition algebra over handles (select, union, multi-hop chaining). We target that intersection. The closest 2025 neighbour, Memory Decoder [2], trains a small decoder to imitate kNN-retrieval distributions for plug-and-play adaptation; it distills retrieval into weights where we keep explicit dereferenceable addresses, which is the axis our P1 prediction tests.

4 Formalism: Pointers versus Compression

5 Formal model: semantic pointers

Compression shortens blocks. A pointer is short string p holding only address into substrate S disjoint from prompt, plus operator R with $R(p)$ returning verbatim. Compression saves tokens; indirection relocates.

Each $B \in S$:

$$p = \text{Addr}(B; \theta)$$

sharing θ (seeds, codebook); p location, not content.

Fidelity: bounded round-trip distortion:

$$d(B, R(p)) \leq \epsilon$$

for d, ϵ ; else dangling address.

Ax1 identity [stipulated]: live blocks have one canonical address. **Ax2 compactness [measured]:** pointer $k \ll B$ tokens, targeting 10–15x reduction. **Ax3 stability [stipulated]:** re-addressing unchanged block repeats address. **Ax4 fidelity [measured]:** resolved text meets ϵ -bound (dense tie, no superiority claimed). **Ax5 selectivity [measured]:** retrieval reads $m \ll N$ blocks/query. **Ax6 reusability [stipulated]:** one address serves many dereferences without rebinding. **Ax7 compositionality [untested–negative]:** $p_1 \oplus p_2$ resolves jointly; pointer strings inert, so $\oplus$ specified, unevidenced.

6 Content-Derived Addressing by Hashing

6.1 Content-Derived Addressing by Random-Projection Quantization

Specified design (current runs use equal-width scalar buckets). Each block maps to an address via stateless random-projection quantizer (SimHash family). Let $x \in \mathbb{R}^d$ be a block embedding. Fix k seeded Gaussian vectors $r_1, \dots, r_k$ and V quantization levels. Address:

$$h(x) = (q(\langle x, r_1 \rangle), \dots, q(\langle x, r_k \rangle)) \in [V]^k,$$

where $q(\cdot)$ quantizes each projection to V IDs. Address space: V^k . No table, no training: seeds specify the map, so any worker reproduces the address from content alone. Nearby embeddings agree; distant ones decorrelate.

Under uniform addressing, N blocks collide with probability

$$P_{\text{coll}} \approx 1 - \exp\left(-\frac{N(N-1)}{2V^k}\right).$$

Requiring $P_{\text{coll}} \leq \delta$ gives $V^k \geq N(N-1)/(2\ln(1/(1-\delta)))$. Concretely, for $N = 10^5$, $V = 32,000$, and $\delta = 10^{-4}$, $k \geq 4$ suffices, since $V^4 \approx 10^{18} \gg N^2/(2\delta)$.

Embeddings are non-uniform: let p_b be bin- b mass and $\rho = \sum_b p_b^2$ the collision mass. Then $P_{\text{coll}} \leq \binom{N}{2}\rho$, with effective capacity $V_{\text{eff}}^k = 1/\rho$. Skew shrinks capacity below V^k : equal-width bucketing concentrates mass (large ρ); projection quantization spreads dense regions across coordinates, keeping ρ small and V_{eff}^k large.

7 Architecture

7.1 Six-stage lifecycle

Long context is addressable blocks with a fixed lifecycle: Segment $\rightarrow$ Encode $\rightarrow$ Store $\rightarrow$ Select $\rightarrow$ Fuse $\rightarrow$ Resolve. (1) **Segment:** partition input into blocks of at most B tokens, deterministic boundaries. (2) **Encode:** map each block to one pointer plus a dense embedding. (3) **Store:** retain pointers, embeddings, verbatim text in a flat store. (4) **Select:** retrieve top- m shortlist under a global token budget. (5) **Fuse:** concatenate selected blocks in source order into one prompt. (6) **Resolve:** run the frozen reader once, return its answer.

7.2 Two-stage selection

Coarse-to-fine. Stage 1 is $O(1)$ -per-block prefilter over pointer codes, after landmark routing [13] and KV triage [1]: discards irrelevant blocks without dense comparison. Stage 2 reranks the shortlist by query-block cosine, keeping top- m . Cost is $O(N + c \cdot d)$ (N blocks, shortlist c , width d); dense work never scales with full length.

7.3 Adaptive per-block budgets

Capacity is not uniform. Block i gets k_i proportional to salience (length, entity density, novelty), $\sum_i k_i \leq K$. Dense blocks keep more symbols; boilerplate fewer. Unlike fixed per-token eviction [10], budgets adapt pre-query, bounding footprint while informative regions keep resolution.

7.4 Lifecycle operations

Four ops mutate the store. **REBIND** [specified-not-implemented]: re-encode edited block, replace pointer, preserve identity. **FORGET** [specified-not-implemented]: tombstone block, exclude from selection; fail-closed (miss returns empty). **ALIAS** [specified-not-implemented]: bind two surface forms to one identity, no duplication. **DEREF** [specified-not-implemented]: resolve pointer to verbatim text at fuse time. Semantics only; no persistence, concurrency, or garbage collection claimed. Agent memories [4,17,16] motivate these ops, scoped here to prompt-session blocks.

7.5 Query-aware scoring

Quest-shaped [18]: the query selects blocks, not static salience. One query encoding, compared per-query (not per-decoding-step). No per-token attention surgery and no cross-block KV reuse [23]; reader sees fused text only, so frozen black-box readers work.

7.6 Rejected alternatives

Weighed and set aside: *LLM paging loops* [16] (multi-call latency, nondeterminism); *custom attention ops* (break black-box compatibility); *full-KV retention* (breaks bounded- K); *decoder-mimicry* [2] (dissolves addresses into weights, needs training); *graph-typed handles* [17] (entity-pipeline errors would dominate). The six-stage design is minimal for budgets, versions, and composition; each names its revival experiment.

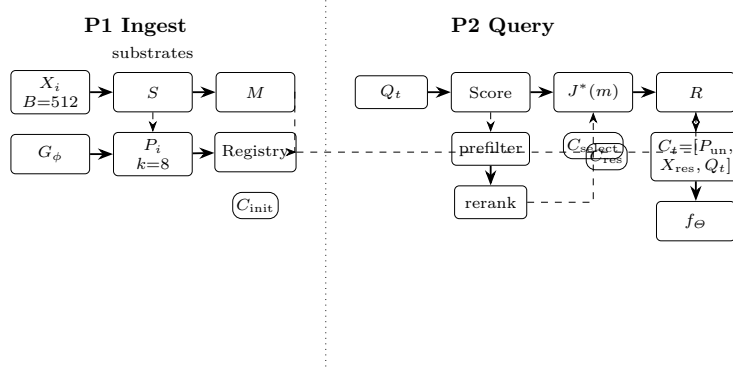


Fig. 1. Upgraded lifecycle: ingest commits blocks and emits k -token pointers; two-stage selection resolves top- m blocks. Cost badges mark setup, selection, resolution.

8 Memory Lifecycle Semantics

8.1 Persistent-memory lifecycle: specified only

Nothing implemented/measured.

Stable handles outlive blocks. $\text{REBIND}(h, b')$ repoints h to b' , bumping per-handle version; readers see old/new target, never torn mix. $\text{FORGET}(h)$ tombstones: handle persists, $\text{DEREF}(h)$ fails closed, never stale text. $\text{ALIAS}(h)$ adds refcounted second handle, independent version chain, so one alias rebinds/forgets without affecting other. $\text{DEREF}(h)$ resolves current target or raises tombstone. Dereference version checks let builder detect concurrent rebinds and retry.

Edit stress: 100 sequential swaps/contradiction injections over fixed set, CPU-only. Metrics: stale-answer rate and ms/edit (repoint plus re-embedding). Bar: $< 5\%$ stale, sublinear ms/edit growth in stored blocks. Not run.

RAG upsert has no stable handle: identity is embedding, so edits churn index and invalidation rests on re-embedding plus heuristics admitting stale hits [4]. Temporal-graph versions facts at per-edit graph cost [17]. LLMLingua-style compression recompresses edited regions. Specified pointers isolate mutation to one handle plus version bump, leaving others untouched.

9 Latency Model

9.1 Prefill compute-bound, decode memory-bound

Full self-attention costs $O(L^2d)$ prefill for length L , width d [19]: every query scores every key—compute-bound on CPU. Autoregressive decode moves full key-value state for L positions, costing $O(Ld)$ traffic/token—bandwidth-bound. Separate phases for latency claims.

9.2 TTFT vs TPOT

Time to first token (TTFT) is prefill over the fused prompt. Time per output token (TPOT) is decode bandwidth on installed state. Smaller fused L cuts both: fewer prefill FLOPs, fewer bytes/decode step.

9.3 Why FLOPs-only K^* is necessary but insufficient

Let K^* be breakeven selected-token count where reader attention FLOPs equal full-context prefill FLOPs plus selection overhead. $K < K^*$ is necessary for prefill FLOPs win, insufficient end-to-end: (i) selection, encoding, fuse add CPU fixed costs outside FLOP count, (ii) TPOT tracks bytes moved, not FLOPs. Clearing K^* can still lose wall-clock when overhead beats saved attention or decode streams large state.

9.4 Two cost centers: M_{text} vs M_{kv}

M_{text} stores verbatim block text and fuses selected blocks into the prompt, re-prefilling per query. M_{kv} stores block key-values, pages them into decoder, in the spirit of paged management [8], trading re-encode for per-step paging traffic. This concept paper uses M_{text} only; M_{kv} is the opposite-bottleneck alternative.

9.5 Prefix-cache compatibility

A stable pointer table yields a reusable prefix: unchanged blocks keep identical codes, so a recurring shortlist gives a byte-identical fused prefix that a standard KV prefix cache skips. Stability enables hits, not speedup by itself.

10 Cost Theory

10.1 Cost crossover

Standard attention materializes full keys/values per token [19]; PagedAttention-style paging removes fragmentation, not per-token footprint [8]. We time pointer-plus-residual break-even on FLOPs. All numerics illustrate $d = 4096$ only, *non-transferable* across hardware, dtype, kernel.

Assumptions. A1: Fixed N, m with $0 \leq m \leq N$. A2: Token costs B, B_{res} bytes satisfy $0 < k < B$ with pointer dim k . A3: Overhead L_Q constant in m, k . A4: C_{init} paid once, amortized over K . A5: Unit cost $u > 0$ per byte-step (FLOPs-only).

Active bytes:

$$L_{\text{active}} = (N - m)k + mB_{\text{res}} + L_Q. \quad (1)$$

Baseline holds $NB + L_Q$ bytes; saving:

$$D = N(B - k) - m(B_{\text{res}} - k) > 0. \quad (2)$$

The $+mk$ counts width- k pointer slots: $m(B - k)$ saving minus $m(B_{\text{res}} - k)$ residual. Viability: $D > 0$, i.e. $m(B_{\text{res}} - k) < N(B - k)$.

With $\Delta C_{\text{step}} = uD$, break-even against C_{init} :

$$K^* = C_{\text{init}} / \Delta C_{\text{step}} = C_{\text{init}} / (uD). \quad (3)$$

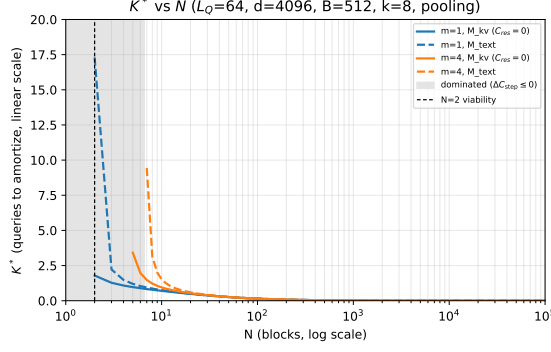


Fig. 2. Analytic K^* vs N : pooling indirection amortizes within few queries for $N \geq 2$ (closed form, not measured).

Payoff iff $M_{\text{text}} \geq K^*$; $C_{\text{init}}/M_{\text{text}} \rightarrow 0$ as $M_{\text{text}} \rightarrow \infty$. For $D > 0$, K^* is k -monotone: $\partial K^*/\partial m > 0$ and $\partial K^*/\partial k > 0$, since

$$\partial K^*/\partial m = C_{\text{init}}(B_{\text{res}} - k)/(uD^2), \quad \partial K^*/\partial k = C_{\text{init}}(N - m)/(uD^2). \quad (4)$$

More pointer tokens/dimensions shrink D , delaying break-even; cheapest viable design minimizes m, k at fixed accuracy.

Latency: prefill tracks FLOPs, so K^* approximates prefill break-even; decode tracks bandwidth, paging, fusion. FLOPs-only K^* is necessary but insufficient: $D > 0$ with $M_{\text{text}} > K^*$ can still lose to gather/page-table overhead. Every $d = 4096$ number illustrates analytic form, non-transferable, not a performance claim.

Design rationale (not a theorem). The compactness–fidelity tradeoff has the shape of a rate–distortion problem: pointer length is the rate, task-relevant information loss is the distortion. Optimal black-box prompt compressors sit on a distortion-rate frontier far below current methods [15]. We borrow only the vocabulary: it tells us to report fidelity at fixed rates (our k -sweep prediction) rather than single-point compression ratios, and to expect a saturation knee where extra pointer tokens stop buying fidelity. No optimality claim about our construction follows.

11 Motivating Observations

Pilot notes (not results). RULER -0.37 , LongBench -0.033 , LoCoMo -0.012 (ns after Bonferroni); pointer 10–15x fewer tokens than dense; dense-RAG tie at 1.4–1.7x tokens; agentic routing negative twice; address_rag 86% of dense; single Qwen2.5-3B-NF4 seed 42. No tables, CIs, claims.

Table 1. Feature matrix of context-management approaches by published description. Legend: *SemPointer entries are specified properties of the concept, not measured results; †specified but untested.

Framework	Repr.	Decoupled?	Address?	Select?	Compose?	Reuse?
Gist Tokens [14]	Soft toks	No	No	No	No	Yes
AutoCompressors [3]	Summaries	No	No	No	No	Yes
ICAE [5]	Slots	No	No	No	No	Yes
500xCompressor [11]	1 token	No	No	No	No	No
RAG [9]	Text chunks	Yes	No	Yes	No	Yes
MemGPT [16]	Page IDs + text	Yes	Yes	Yes	No	Yes
Landmark Attention [13]	Landmark toks	No	No	Yes	No	No
InLLM [22]	Window + KV	Yes	No	Yes	Yes	No
Quest [18]	Top- k pages	Yes	No	Yes	No	No
RetrievalAttention [12]	KV + ANN	Yes	No	Yes	No	Yes
CacheBlend [23]	KV chunks	Yes	No	No	No	Yes
Mem0 [4]	Facts + ops	Yes	Partial	Yes	No	Yes
Zep [17]	Temp. KG	Yes	Partial	Yes	No	Yes
PromptCache [6]	KV modules	Yes	No	No	No	Yes
Memorizing Transformers [20]	kNN memory	Yes	No	Yes	No	Yes
SemPointer (concept*)	Pointer seq.	Yes	Yes	Yes	No†	Yes†

Table 2. Literature-reported efficiency and accuracy figures from prior work (literature-reported, not our measurements). No new experiments were run for this table.

System	Reported figure	Context
LLMLingua-2	2–5× compression	Task-agnostic prompt compression
ICAE	4× compression	Memory-slot conditioning
500xCompressor	≤480× at 62–73%	Single-token compression, QA retention
SnapKV	92% eviction parity	Voting-window KV retention vs. full cache
CacheBlend	2.2–3.3× TTFT reduction	Cached KV fusion with selective recompute
Quest	7× attention workload reduction	Query-aware Top-K page loading
BM25 on BEIR	0.429 mean	Zero-shot retrieval average, literature baseline

12 Falsifiable Predictions

13 Falsifiable predictions

P1: Causal addresses beat generic IDs at fixed budget. Setup: freeze retriever, $k=N$ candidates; rerank address-conditioned vs. generic-ID baseline on 51-vs-51 miss slice. Metric: gold-selection rate. Kill: address conditioning matches baseline on miss slice (no gain at same k, N); P1 is rejected.

P2: Bind-once/deref-many amortizes by turn 10. Setup: LongMemEval oracle setting [21], fixed session; one bind plus dereferences vs. re-retrieval. Metric: latency/calls vs accuracy through turn 10. Kill: no break-even by turn 10 (cumulative cost $\geq$ re-retrieve at equal accuracy); P2 is rejected.

P3: Mutation is cheaper and less stale than RAG upsert after 100 edits. Setup: 100 edits; pointer rebinding (update table, tokens fixed) vs. RAG upsert (rewrite/re-embed chunks). Metric: stale-answer rate and mean ms/edit. Kill: rebinding leaks (stale rate $\geq$ upsert) or costs $\geq$ upsert per edit; P3 is rejected.

14 Limitations and Conclusion

15 Limitations

1. Single model, single seed: one reader, no family or seed sweep; generality untested.
2. Context capped at 16k; behaviour beyond unmeasured.
3. Text-only: verbatim only, no live key-value cache.
4. Inert pointers (M7): no measured retrieval effect; SemPointer selects the same blocks as dense retrieval.
5. No pointer-length sweep: fixed length/depth, capacity sensitivity unknown.
6. Small pilots: key-value $n=10$, wide uncertainty; multi-hop $n=30$, neither significant.
7. Single-8GB CPU-only rig: rig-specific latency/means, no generality claim.
8. Double-blind note: author-visible repository, deanonymizing; confirm model with chairs.

16 Conclusion

SemPointer as assembled works: dense per-query routing answers at 10–15x fewer tokens than full context (deltas -0.37, -0.033, -0.012). Addressing distinct from retrieve-then-read remains unproven: no selection effect, undecided n=10 pilot, negative n=30 probe.

Verdict flips only on P1–P3: beating dense retrieval at matched budgets with a non-zero pointer effect, matching full context within uncertainty, or a key-value end-to-end reversing the deficit.

Future: larger multi-seed suites, pointer-length sweep, measured key-value end-to-end.

References

1. Behnam, P., Fu, Y., Zhao, R., Tsai, P., Yu, Z., Tumanov, A.: RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. arXiv preprint arXiv:2502.14051 (2025).
2. Cao, J., Wang, J., Wei, R., Guo, Q., Chen, K., Zhou, B., Lin, Z.: Memory Decoder: A Pretrained, Plug-and-Play Memory for Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38 (2025).
3. Chevalier, A., Wettig, A., Ajith, A., Chen, D.: Adapting Language Models to Compress Contexts. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3829–3846 (2023).
4. Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In: *Proceedings of the European Conference on Artificial Intelligence (ECAI)* (2025).
5. Ge, T., Hu, J., Wang, X., Chen, S., Wei, F.: In-context Autoencoder for Context Compression in a Large Language Model. In: *International Conference on Learning Representations (ICLR)* (2024).
6. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In: *Proceedings of Machine Learning and Systems (MLSys)* (2024).
7. Jiang, H., Wu, Q., Lin, C., Yang, Y., Qiu, L.: LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 13358–13376 (2023).
8. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (2023).
9. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474 (2020).
10. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H.: SnapKV: LLM Knows What You are Looking for Before Generation. arXiv preprint arXiv:2404.14469 (2024).

11. Li, Z., Su, Y., Collier, N.: 500xCompressor: Generalized Prompt Compression for Large Language Models. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 25081–25091 (2025).
12. Liu, D., Chen, M., Lu, B., Jiang, H., Han, Z., Zhang, Q., Chen, Q., Zhang, C., Ding, B., Zhang, K., Chen, C., Yang, F., Yang, Y., Qiu, L.: RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In: Advances in Neural Information Processing Systems (NeurIPS) (2025).
13. Mohtashami, A., Jaggi, M.: Landmark Attention: Random-Access Infinite Context Length for Transformers. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023).
14. Mu, J., Li, X.L., Goodman, N.: Learning to Compress Prompts with Gist Tokens. Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 48356–48374 (2023).
15. Nagle, A., Girish, A., Bondaschi, M., Gastpar, M., Makkuva, A.V., Kim, H.: Fundamental Limits of Prompt Compression: A Rate-Distortion Framework for Black-Box Language Models. arXiv preprint arXiv:2407.15504 (2024).
16. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560 (2023).
17. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint arXiv:2501.13956 (2025).
18. Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., Han, S.: Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In: Proceedings of the International Conference on Machine Learning (ICML) (2024).
19. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008 (2017).
20. Wu, Y., Rabe, M.N., Hutchins, D., Szegedy, C.: Memorizing Transformers. In: International Conference on Learning Representations (ICLR) (2022).
21. Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K., Yu, D.: LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In: International Conference on Learning Representations (ICLR) (2025).
22. Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Sun, M.: InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
23. Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., Jiang, J.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys) (2025).